

Optimized Cytoscape WebGL and Sigma Evaluation

Current Chrome evidence, product-feature parity, grouping architecture, and implementation tradeoffs

**Continue with Sigma as the leading renderer candidate while retaining the
renderer-neutral grouping and entity model.**

60.1 FPS

Sigma feature-rich pan

1.20 sec

Sigma target ready

554 MiB

Sigma added working set

328 MiB

Sigma added graphics
memory

Primary test environment: Chrome 152 | Five runs per renderer | Complete memory measured September 14, 2026

Executive summary

This report compares Cytoscape 3.34.3 WebGL and Sigma 4.0.0-beta.5 after both implementations were optimized for the MicrobeTrace network workload. The primary evidence is a set of repeated Chrome 152 runs on the same datasets, with the same layout inputs, full-detail edge workload, interaction probes, and product requirements.

Sigma is the leading candidate for the feature-rich MicrobeTrace renderer:

- On the sparse representative MT networks, the two renderers have broadly comparable startup and navigation medians. Cytoscape synchronizes selection faster.
- On the 1,600-node/3,200-link feature-rich workload, Sigma sustained approximately 60 FPS for pan and zoom in every sample. Cytoscape medians were approximately 40 FPS for pan and 46 FPS for zoom.
- Sigma reached the feature-rich target view in a 1.20 second median versus 3.65 seconds for Cytoscape, expanded grouped data faster, and used substantially less total Chrome and graphics memory in the isolated memory run.
- Cytoscape retained advantages in selection latency, group-collapse latency, native compound-graph semantics, and recovery to a non-WebGL Canvas renderer.

The renderer choice does not need to determine the scientific hierarchy. MicrobeTrace now has a renderer-neutral grouping/entity model that can drive networks, tables, filters, maps, statistics, exports, and saved sessions. Cytoscape can project that model as compound nodes; Sigma can project it as hulls and collapsed aggregates.

Evaluation design

Compared implementations

Area	Optimized Cytoscape WebGL	Optimized Sigma
Library	Cytoscape 3.34.3	Sigma 4.0.0-beta.5 with Graphology 0.26.0
Primary rendering	Cytoscape WebGL renderer	Sigma WebGL renderer
Shared layout	MT-owned sparse D3-force backbone	Same MT-owned sparse D3-force backbone
Feature glyphs	MT Canvas overlay above WebGL nodes	Custom Sigma WebGL node program
Group display	Native compound nodes plus shared collapsed projection	MT Canvas hulls plus shared collapsed projection
Geographic display	Shared projected coordinates and Canvas reference overlay	Same
Edge workload	All expected edges displayed during interaction probes	Same
Selection optimization	Cytoscape selection/style update	Partial Sigma refresh of changed nodes and affected edges, without spatial re-indexing
Failure behavior	Falls back to Cytoscape Canvas	Recreates Sigma WebGL while retaining shared state

The Cytoscape WebGL texture and batching configuration is tuned for this workload: 4,096 texture size, 48 texture rows, 16,384 elements per batch, 16 textures per batch, and pixel ratio 1. Sigma uses viewport-aware edge level of detail outside the full-detail probes and partial graph refreshes for state-only changes.

Workloads

Workload	Nodes	Displayed links	Feature content	Purpose
MT distance network	1,600	537	Committed TN93 distance data filtered at 0.015	Same real MT network through both renderers
MT distance + epidemiologic network	1,600	596	Same distance data plus 59 epidemiologic links	Same representative mixed-link workload
Feature-rich MT network	1,600	3,200	40 case groups, 160 specimens, 800 mixed-value donuts, QC and uncertainty on every node, geographic coordinates on every node	Expose renderer architecture and customization differences

Measurement rules

- Chrome 152 headless is the primary benchmark environment; Edge 152 is the secondary compatibility environment.
- Each primary performance result contains five interleaved samples per renderer.
- The requested renderer must remain in WebGL. Silent fallback fails the sample.
- Resident and drawn element counts must match the scenario before timing is accepted.
- Interaction timing begins after a 500 ms idle period and two presented animation frames.
- Pan and zoom use an 800 ms continuous camera probe.
- Values in the tables are **median [minimum-maximum]** across five samples.
- Layout time is reported but not treated as a renderer score because the shared algorithm runs inside separate application launches.
- Selection measures synchronous state propagation and redraw scheduling, not visual completion of the following frame.
- The isolated memory lane starts a fresh Chrome process for every renderer sample and records the entire seven-process Chrome tree at five workflow checkpoints.
- Windows process counters provide working set and private committed memory. Windows GPU counters provide dedicated, shared, and total committed graphics memory for those same Chrome process IDs.
- Working set includes shared pages that can appear in more than one process. Private committed memory is therefore included as a second whole-browser measure. "Peak observed" means the largest of the five checkpoints, not a continuously sampled maximum.

Completed Chrome coverage

Coverage	Samples/tests	Result
Representative MT comparison	20 measurements: 2 datasets x 2 renderers x 5 samples	Passed
Feature-rich comparison	10 measurements: 2 renderers x 5 samples	Passed
Isolated complete memory	10 fresh Chrome processes: JavaScript heap, full process tree, and GPU allocations	Passed
Feature parity and accessibility contract	2 tests	Passed
Saved-session round trip	1 test	Passed
Forced WebGL recovery	2 tests	Passed

Every performance sample retained 1,600 resident nodes, displayed the expected full-detail edge count, and reported the requested WebGL renderer as active.

Representative MicrobeTrace results

Chrome 152 headless, September 11, 2026. Performance run ID: 2026-09-11T16-15-14-940Z.

Fixture	Renderer	Target ready (ms)	Shared layout (ms)	Renderer create (ms)	Pan FPS / p95 gap (ms)	Zoom FPS / p95 gap (ms)	Select (ms)	Full edges
Distance only	Cytoscape WebGL	1,222.1 [917.0-1,434.0]	1,659.8 [1,237.0-1,909.0]	131.1 [118.4-235.0]	55.28 [3.66-60.17] / 16.9 [16.8-619.1]	55.62 [46.14-60.44] / 16.8 [16.8-100.0]	0.8 [0.7-1.0]	537
Distance only	Sigma WebGL	1,185.3 [809.4-2,158.5]	1,570.3 [1,171.3-1,925.9]	139.9 [133.1-1,524.8]	52.96 [47.23-60.15] / 16.9 [16.8-100.0]	54.09 [46.83-60.18] / 16.9 [16.8-100.0]	6.4 [5.7-6.9]	537

Fixture	Renderer	Target ready (ms)	Shared layout (ms)	Renderer create (ms)	Pan FPS / p95 gap (ms)	Zoom FPS / p95 gap (ms)	Select (ms)	Full edges
Distance + epi	Cytoscape WebGL	1,227.5 [799.3-1,815.5]	1,798.2 [1,309.9-2,474.4]	130.5 [114.9-140.0]	51.94 [43.47-60.14] / 16.8 [16.8-121.9]	51.86 [44.55-60.50] / 33.3 [16.7-116.7]	0.8 [0.7-1.0]	596
Distance + epi	Sigma WebGL	1,310.9 [931.9-3,325.9]	1,773.3 [1,423.7-4,010.3]	138.6 [132.8-381.9]	49.48 [21.01-58.34] / 33.3 [16.8-209.3]	49.21 [33.27-59.02] / 16.9 [16.8-183.3]	6.3 [6.2-16.5]	596

The representative networks are relatively sparse. Their medians do not establish a decisive startup or navigation winner, and both renderers experienced occasional browser-scheduling outliers. Cytoscape's sub-millisecond selection synchronization is consistently faster, while Sigma's 6-7 ms selection medians remain within an interactive response budget.

Feature-rich results

Chrome 152 headless, September 11, 2026. Performance run ID: 2026-09-11T16-21-13-287Z.

Renderer	Feature drawing	Target ready (ms)	Renderer create (ms)	Pan FPS / p95 gap (ms)	Zoom FPS / p95 gap (ms)	Select (ms)	Composite export (ms)	Collapse 40 groups (ms)	Expand (ms)
Cytoscape WebGL	MT Canvas glyph overlay	3,645.8 [3,546.9-4,015.7]	252.3 [190.8-349.6]	40.23 [35.72-43.23] / 33.4 [33.4-33.6]	45.95 [42.93-60.13] / 33.4 [16.9-33.5]	2.2 [1.9-2.4]	11.9 [11.3-41.4]	60.0 [52.4-71.9]	316.6 [252.9-487.6]
Sigma WebGL	Custom WebGL node program	1,198.4 [1,156.5-2,681.3]	175.5 [174.4-1,640.9]	60.14 [60.12-60.15] / 16.8 [16.8-16.8]	60.25 [60.23-60.29] / 16.8 [16.8-16.8]	13.2 [12.3-15.6]	8.8 [8.2-10.7]	93.4 [88.5-102.1]	191.2 [181.4-211.5]

The feature-rich workload is where the architectural difference is visible:

- Sigma maintained refresh-rate pan and zoom in every sample.
- Cytoscape's Canvas feature overlay reduced its median navigation rate even though the underlying graph remained in WebGL.
- Sigma had lower median target-ready, renderer-create, export, and expansion times.
- Cytoscape had lower selection and collapse latency.
- Sigma's first sample contains a cold-start creation outlier, but its steady interaction measurements remain narrow.

The custom Sigma program renders up to four visible donut sectors along with QC and uncertainty signals. Additional sectors are combined into the fourth visible sector; the complete composition remains available to the renderer-neutral model, accessibility text, statistics, export metadata, and saved data. If the custom shader cannot initialize, Sigma uses the semantically equivalent Canvas glyph path.

Complete isolated Chrome and graphics memory

Chrome 152 headless on Windows, September 14, 2026. Five fresh browser processes were used per renderer, with renderer order alternated between rounds. Each run tracked the browser, two renderer processes, two utility processes, the GPU process, and the crash handler. Garbage collection was forced before the initial reading and at later checkpoints. Result IDs span 2026-09-14T15-55-34-090Z through 2026-09-14T16-06-59-073Z.

Whole Chrome process tree

Renderer	Initial working set (MiB)	Final working set (MiB)	Added working set (MiB)	Peak observed working set (MiB)	Added private memory (MiB)
Cytoscape WebGL + Canvas glyphs	1,029.8 [1,026.8-1,047.3]	2,186.1 [2,061.4-2,213.5]	1,153.6 [1,034.6-1,184.6]	2,186.1 [2,076.9-2,213.5]	1,131.2 [1,010.7-1,160.9]
Sigma custom WebGL glyphs	1,041.2 [1,026.5-1,073.9]	1,595.0 [1,369.0-1,667.5]	553.8 [295.1-626.5]	1,596.8 [1,483.3-1,667.5]	531.4 [266.0-601.8]

The initial Chrome baseline was similar for both renderers. After the complete workflow, Sigma added approximately 599.8 MiB less working-set memory and 599.8 MiB less private committed memory at the median. Those are reductions of approximately 52% and 53%, respectively.

Graphics memory

Renderer	Initial resident graphics memory (MiB)	Final resident graphics memory (MiB)	Added resident graphics memory (MiB)	Peak observed resident graphics memory (MiB)	Final committed graphics memory (MiB)
Cytoscape WebGL + Canvas glyphs	125.4 [121.9-138.5]	649.2 [512.6-657.9]	510.7 [390.8-529.8]	656.1 [535.5-684.3]	650.2 [513.6-658.9]
Sigma custom WebGL glyphs	141.1 [139.7-162.6]	470.9 [280.2-474.3]	328.2 [117.6-333.3]	470.9 [282.2-474.3]	471.9 [281.2-475.3]

This Windows host uses shared graphics memory, so dedicated GPU memory was zero and the resident total is the shared allocation. Sigma added approximately 182.5 MiB less resident graphics memory at the median, a reduction of approximately 36%.

JavaScript heap diagnostic

Renderer	Initial heap (MiB)	Final heap (MiB)	Incremental heap (MiB)
Cytoscape WebGL + Canvas glyphs	195.0 [195.0-195.0]	278.0 [277.6-279.5]	83.1 [82.6-84.5]
Sigma custom WebGL glyphs	195.0 [194.9-195.1]	236.2 [236.1-237.5]	41.2 [41.1-42.5]

JavaScript heap remains useful for diagnosis, but the whole-browser and graphics figures above are the primary memory comparison. The newer complete-memory run confirms the earlier heap-only direction: Sigma used approximately 50% less incremental JavaScript heap at the median.

Product and architectural comparison

Requirement	Optimized Cytoscape WebGL	Optimized Sigma	Current status
Mixed-value donut nodes	MT Canvas glyph overlay	Custom WebGL node program with Canvas fallback	Semantic, pixel, accessibility, export, and scale contracts pass
Cluster hulls and grouping	Native compound groups plus shared collapsed projection	MT Canvas hulls plus shared collapsed projection	Membership, group selection, group drag, statistics, collapse/expand, and export pass
QC and uncertainty	Shared Canvas glyph overlay	Custom WebGL node program with Canvas fallback	All 1,600 feature-scale nodes and 40 collapsed aggregates validated
Semantic zoom and collapse	Aggregate Cytoscape nodes; compounds restored on expand	Aggregate Sigma nodes; edge LOD remains independent	Source data, selection, focus, statistics, and saved state preserved
Geographic overlay	Shared Web Mercator positions and Canvas reference layer	Same	Position ordering, group centroids, export, and session restore pass
Edge rendering	All required links rendered during comparison probes	Same	537, 596, and 3,200-link workloads validated without silent omission
Selection and highlighting	Native selection plus shared keyboard/group behavior	Partial changed-element refresh plus shared keyboard/group behavior	Node/group selection, neighbor emphasis, cross-view continuity, focus, and live status pass
Export fidelity	Composite PNG and parseable SVG containing WebGL and overlays	Same	Expanded/collapsed graph scope and feature metadata pass at scale
Session restoration	Restores grouping, collapse, selection, styles, features, and camera	Same, including explicit edge-detail mode	Real .microbetrace save/reload round trip passes
Accessibility	Renderer-neutral text alternative, focus target, keyboard actions, and live status	Same	Automated contracts pass; manual assistive-technology validation remains
WebGL failure recovery	Falls back to Cytoscape Canvas while retaining represented state	Recreates Sigma after context loss while retaining graph, camera, and selection	Forced recovery contracts pass, including two consecutive Sigma losses
Browser compatibility	Chrome 152 full run; Edge 152 secondary smoke coverage	Same	Both current target browsers pass; Firefox and Safari/WebKit are outside current scope
Complete memory	Full Chrome process tree and Windows GPU allocations	Same	Five fresh processes per renderer passed; Sigma used less added process and graphics memory

The feature-rich SVG export embeds a lossless raster composite so WebGL and Canvas layers are represented consistently. A vector-native SVG implementation is needed only if editable vector output is a confirmed product requirement.

Compound nodes and the MT grouping model

Cytoscape has native parent/child graph semantics; Sigma does not. That difference matters only if parent/child containment must be part of the graph engine itself.

The current implementation separates scientific grouping from rendering:

Concern	MT grouping/entity model owns	Renderer owns
Hierarchy	Entity levels, parent identifiers, and membership	Visual projection of the hierarchy
Aggregation	Counts, feature composition, geographic centroid, and group statistics	Drawing aggregate nodes, labels, or hulls
Collapse state	Which groups are collapsed and which source nodes they represent	Displaying the projected nodes and links
Selection	Source entities selected by a node, aggregate, hull, table row, or map feature	Visual highlight and pointer interaction
Persistence	Renderer-neutral group, collapse, selection, and camera state	Translation to and from the renderer camera

This architecture allows Cytoscape to use compound nodes and Sigma to use hulls/collapsed aggregates without changing the scientific model. The same hierarchy can be reused by tables, filters, maps, statistics, exports, and saved sessions. Native compound support therefore remains a Cytoscape implementation advantage, but it is not by itself a reason to bind MicrobeTrace's domain model to

Cytoscape.

Implementation complexity and risk

Dimension	Cytoscape WebGL	Sigma
Integration shape	Extends the existing 7,647-line 2D component, so renderer-specific code is difficult to isolate	Dedicated 1,736-line adapter plus a 165-line custom WebGL node program
Shared MT architecture	Uses 1,055 lines of shared grouping, feature, geography, export, view-state, and overlay modules	Same
Custom drawing	Canvas feature and geography overlays above Cytoscape WebGL	Custom WebGL feature glyph plus Canvas hull and geography layers
Recovery cost	Product-owned WebGL-to-Canvas transition	Product-owned renderer recreation
API risk	Cytoscape release is stable; its WebGL renderer and tuning surface are provisional	Sigma 4 dependency is beta
Maintenance tradeoff	Lower incremental implementation but greater coupling to the legacy 2D component	Larger explicit renderer surface with clearer extension and isolation points

Line counts are a maintenance inventory, not a quality score. Sigma requires more renderer-specific code, but that code is isolated. Cytoscape reuses more of the existing component, but its renderer behavior remains coupled to a much larger legacy surface.

Recommendation

Continue with Sigma as the leading renderer candidate for the next MicrobeTrace network implementation while retaining the renderer-neutral grouping/entity model.

The strongest evidence is the stable feature-rich Chrome result: Sigma maintained approximately 60 FPS while rendering the custom node features in WebGL, used substantially less complete browser and graphics memory, and restored expanded group topology faster. Cytoscape remains competitive on sparse representative networks and is stronger for immediate selection updates, group collapse, native compound semantics, and non-WebGL fallback.

Reproduction commands

Run the same committed representative datasets through both optimized renderers:

```
npm run e2e:renderer-compare:representative
```

Run the feature-rich comparison:

```
npm run e2e:renderer-compare:feature-scale
```

Run isolated forced-GC JavaScript heap, full Chrome process-tree, and GPU memory measurements in fresh Chrome processes:

```
npm run e2e:renderer-memory
```

Run feature, accessibility, session-restoration, and WebGL-recovery contracts:

```
npm run e2e:renderer-contracts
```

The opt-in Performance Baseline GitHub workflow runs these Chrome lanes when `renderer_compare` is enabled and retains the generated Cypress artifacts for 30 days. Edge remains available as the secondary compatibility target through the explicit `:edge:local` commands.

Technical references

[Cytoscape WebGL preview](#) | [Sigma lifecycle and partial refresh](#) | [Sigma custom renderer programs](#)